Project Report: Comprehensive Data Pipeline for E-Commerce Analytics

Author: Nivetha M P

Project: Comprehensive Data Pipeline with Azure Data Factory, Databricks and Dashboard for E-Commerce Analytics

1. Project Objective and Architecture

This capstone project focuses on building a comprehensive data pipeline for Data Engineering and Analytics. The architecture handles data extraction, transformation, and loading (ETL) across a multi-cloud environment.

We primarily aim to use Azure Data Factory for data ingestion, then switch to Databricks with PySpark and Spark-SQL for transformation and analytics. The architecture includes multiple data sources, a NoSQL landing zone, SQL-based materialized views, and a visualization layer. By the end of this project, we have a functional pipeline that provides valuable insights.

Dataset: The project utilizes the "Unlock Profits with E-commerce Sales Data" dataset from Kaggle. It contains data on a variety of sales channels, including Shiprocket and INCREFF, as well as financial information on related expenses and profits. The columns contain data such as SKU codes, design numbers, stock levels, product categories, sizes and colors

2. Implementation Steps and Documentation

Phase 1: Environment Setup and Initial Data Ingestion

1. First, we set up our Azure environment. This involves provisioning an Azure Storage account named capstonestorageac. Within this storage account, a dedicated container named ecommercestorage is created to act as the initial drop-point for our raw files.
2. Next, we set up Azure Data Factory to create our data workflows and data pipeline. The Data Factory instance datafactorycapstoneproj is successfully deployed within the specified resource group.
3. To automate the data ingestion process, we integrate directly with Kaggle's API. A linked service is created in Azure Data Factory using the HTTP protocol to connect securely to the Kaggle dataset URL using API credentials.
4. We create data pipelines in Azure Data Factory to automate the data ingestion process. A "Copy data" activity is configured to pull the compressed dataset from Kaggle and sink it directly into our Azure Blob Storage container (ecommercestorage)

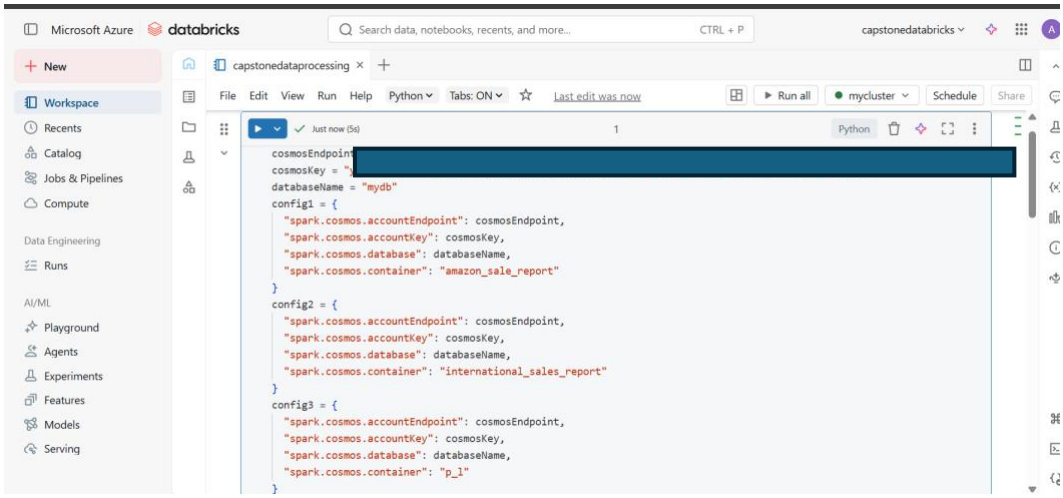
5. The pipeline executes successfully, unzipping and depositing the raw CSV files (such as Amazon Sale Report.csv, International sale Report.csv, etc.) into the blob container.

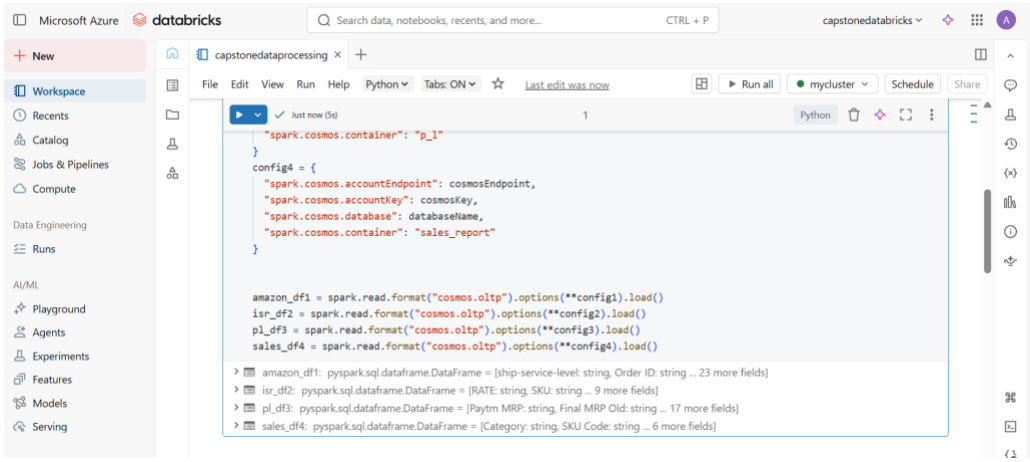
Phase 2: NoSQL Landing Zone Creation

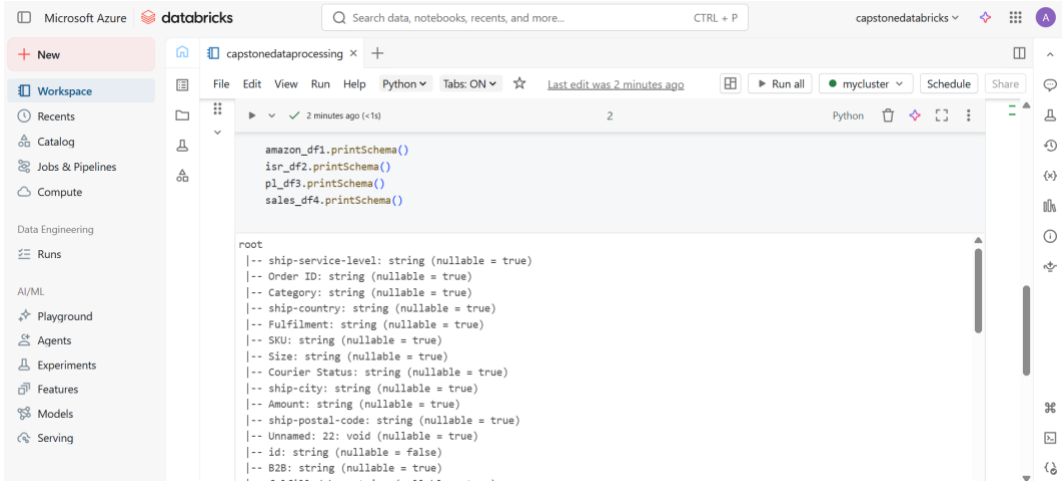
1. We direct ingested data into the NoSQL landing zone, which in our case is Azure Cosmos DB. A Cosmos DB for NoSQL account (cosmosdbcapstone) is provisioned. Inside the Data Explorer, a database (mydb) is created, housing several partitioned containers (amazon_sale_report, international_sales_report, sales_report, and p_l) corresponding to the raw CSV files. This zone acts as a staging area for initial data collection.
2. Returning to Azure Data Factory, we establish linked services pointing to our new Cosmos DB instance. We construct a multi-stage pipeline utilizing sequential "Copy data" activities to move the raw CSV data from Blob Storage into the respective Cosmos DB containers

Phase 3: Data Transformation with Databricks and PySpark

1. We set up a Databricks workspace for data processing and analytics. Once the workspace (capstonedatabricks) is deployed, an all-purpose compute cluster (mycluster) is created with specific worker types and auto-termination parameters to handle the PySpark workloads efficiently
2. To enable seamless communication between Databricks and Azure Cosmos DB, personal access tokens are generated. Crucially, the azure-cosmos-spark Maven library is installed on the Databricks cluster, providing the necessary connectors to read and write Cosmos DB data using PySpark.
3. Inside the Databricks notebook, we connect Databricks to Azure Cosmos DB to read the raw data. Configuration objects are defined with the Cosmos DB endpoint and access keys. The data is loaded into respective PySpark DataFrames (amazon_df1, isr_df2, pl_df3, sales_df4), and their initial schemas are printed for verification.

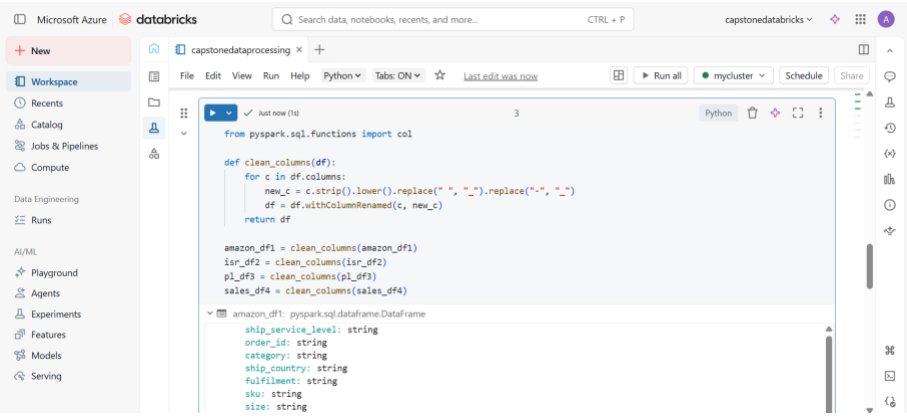




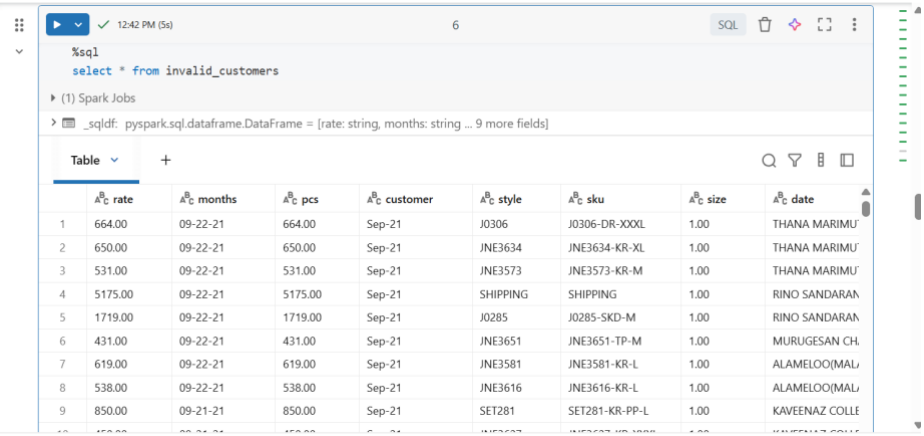


Description: We use PySpark to clean the data, handling missing values, duplicates, and inconsistencies.

Column Standardization: A custom function is applied to strip whitespaces and convert all column headers to lowercase.



Data Cleansing: Orders with invalid, blank, or junk customer names are filtered out and moved to a separate invalid_customers table for auditing.



	rate	months	pcs	customer	style	sku	size	date
1	664.00	09-22-21	664.00	Sep-21	J0306	J0306-DR-XXXL	1.00	THANA MARIMU
2	650.00	09-22-21	650.00	Sep-21	JNE3634	JNE3634-KR-XL	1.00	THANA MARIMU
3	531.00	09-22-21	531.00	Sep-21	JNE3573	JNE3573-KR-M	1.00	THANA MARIMU
4	5175.00	09-22-21	5175.00	Sep-21	SHIPPING	SHIPPING	1.00	RINO SANDARAN
5	1719.00	09-22-21	1719.00	Sep-21	J0285	J0285-SKD-M	1.00	RINO SANDARAN
6	431.00	09-22-21	431.00	Sep-21	JNE3651	JNE3651-TP-M	1.00	MURUGESAN CH
7	619.00	09-22-21	619.00	Sep-21	JNE3581	JNE3581-KR-L	1.00	ALAMELOO(MAL
8	538.00	09-22-21	538.00	Sep-21	JNE3616	JNE3616-KR-L	1.00	ALAMELOO(MAL
9	850.00	09-21-21	850.00	Sep-21	SET281	SET281-KR-PP-L	1.00	KAVEENAZ COLLE

Schema Refinement: Unwanted index and ID columns are dropped.

Type Casting: String amounts and quantities are appropriately cast to Double and Integer types, while dates are converted into formal DateType formats.

```python
#type casting
from pyspark.sql.functions import col, to_date

amazon_df1 = amazon_df1.withColumn("amount", col("amount").cast("double"))
amazon_df1 = amazon_df1.withColumn("qty", col("qty").cast("int"))
amazon_df1 = amazon_df1.withColumn("date", to_date(col("date"), "MM-dd-yyyy"))
```

amazon_df1: pyspark.sql.dataframe.DataFrame = [ship_service_level: string, order_id: string ... 20 more fields]

```python
isr_df2 = isr_df2.withColumn("rate", col("rate").cast("double"))
isr_df2 = isr_df2.withColumn("pcs", col("pcs").cast("int"))
isr_df2 = isr_df2.withColumn("gross_amt", col("gross_amt").cast("double"))
isr_df2 = isr_df2.withColumn("date", to_date(col("date"), "MM-dd-yyyy"))
```

isr_df2: pyspark.sql.dataframe.DataFrame = [rate: double, sku: string ... 7 more fields]

Enrichment: We add any additional information or calculations that enhance the value of the data, such as creating a derived total_revenue column by multiplying amount and qty. Null values are handled (e.g., replacing empty promotion IDs with "No Promotion").

```python
from pyspark.sql.functions import col

mrp_cols = [
    "paytm_mrp", "final_mrp_old", "myntra_mrp", "ajio_mrp",
    "mrp_old", "amazon_mrp", "amazon_fba_mrp",
    "snapdeal_mrp", "flipkart_mrp", "limeroad_mrp"
]

for c in mrp_cols:
    pl_df3 = pl_df3.withColumn(c, col(c).cast("double"))
```

pl_df3: pyspark.sql.dataframe.DataFrame = [paytm_mrp: double, final_mrp_old: double ... 15 more fields]

```python
sales_df4 = sales_df4.withColumn("stock", col("stock").cast("int"))
```

sales_df4: pyspark.sql.dataframe.DataFrame = [category: string, sku_code: string ... 4 more fields]

```python
sales_df4 = sales_df4.withColumn("stock", col("stock").cast("int"))
```

sales_df4: pyspark.sql.dataframe.DataFrame

```
category: string
sku_code: string
size: string
design_no: struct
    : string
color: string
stock: integer
```

```python
sales_df4.select("design_no").show(5,False)
```

(2) Spark Jobs

```
+---------+
|design_no|
+---------+
|{J0001}  |
|{CH213}  |
|{CH211}  |
|{CH210}  |
|{CH208}  |
+---------+
only showing top 5 rows
```

Just now (<1s) 10 Python

```python
from pyspark.sql.functions import col

sales_df4 = sales_df4.withColumn("design_no", col("design_no")[""])
```

sales_df4: pyspark.sql.dataframe.DataFrame
- category: string
- sku_code: string
- size: string
- design_no: string

Just now (<1s) 12 Python

```python
amazon_df1 = amazon_df1.dropDuplicates()
isr_df2 = isr_df2.dropDuplicates()
pl_df3=pl_df3.dropDuplicates()
sales_df4=sales_df4.dropDuplicates()
```

- amazon_df1: pyspark.sql.dataframe.DataFrame = [ship_service_level: string, order_id: string ... 20 more fields]
- isr_df2: pyspark.sql.dataframe.DataFrame = [rate: double, sku: string ... 7 more fields]
- pl_df3: pyspark.sql.dataframe.DataFrame = [paytm_mrp: double, final_mrp_old: double ... 15 more fields]
- sales_df4: pyspark.sql.dataframe.DataFrame = [category: string, sku_code: string ... 4 more fields]

Just now (<1s) 13 Python

```python
#adding an derived column
from pyspark.sql.functions import col

amazon_df1 = amazon_df1.withColumn(
    "total_revenue",
    col("amount") * col("qty")
)
```

- amazon_df1: pyspark.sql.dataframe.DataFrame = [ship_service_level: string, order_id: string ... 21 more fields]

[Shift+Enter] to run and move to next cell

Just now (<1s) 11 Python

```python
amazon_df1 = amazon_df1.dropna(subset=["amount", "qty"])
amazon_df1 = amazon_df1.fillna({
    "promotion_ids": "No Promotion"
})
isr_df2 = isr_df2.dropna(subset=["rate", "pcs", "gross_amt"])
pl_df3 = pl_df3.fillna(0)
sales_df4 = sales_df4.dropna(subset=["stock"])
```

- amazon_df1: pyspark.sql.dataframe.DataFrame = [ship_service_level: string, order_id: string ... 20 more fields]
- isr_df2: pyspark.sql.dataframe.DataFrame = [rate: double, sku: string ... 7 more fields]
- pl_df3: pyspark.sql.dataframe.DataFrame = [paytm_mrp: double, final_mrp_old: double ... 15 more fields]
- sales_df4: pyspark.sql.dataframe.DataFrame = [category: string, sku_code: string ... 4 more fields]

[Shift+Enter] to run and move to next cell
[Ctrl+Shift+P] to open the command palette

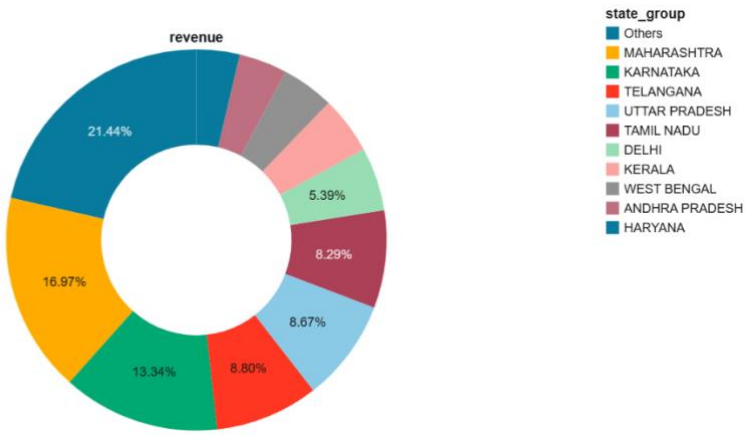
Phase 4: Visualization and Analytics

Description: With clean data, we perform aggregations to uncover business insights. We group the data by shipping state to calculate total revenue, identifying the Top 10 performing states and categorizing the rest as "Others". A pie chart is generated directly within Databricks to visualize this revenue distribution visually.

```python
from pyspark.sql.functions import col, sum, when

# Step 1: Clean amount
df = amazon_df1.withColumn("amount", col("amount").cast("double"))

# Step 2: Total revenue per state
state_df = df.groupBy("ship_state") \
    .agg(sum("amount").alias("total_revenue"))

# Step 3: Get Top 10 states
top10_states = state_df.orderBy(col("total_revenue").desc()).limit(10)

# Step 4: Mark others
top10_list = [row["ship_state"] for row in top10_states.collect()]

final_df = state_df.withColumn(
    "state_group",
    when(col("ship_state").isin(top10_list), col("ship_state")).otherwise("Others")
)
```

```python
# Step 5: Aggregate again
pie_df = final_df.groupBy("state_group") \
    .agg(sum("total_revenue").alias("revenue")) \
    .orderBy(col("revenue").desc())

display(pie_df)
```

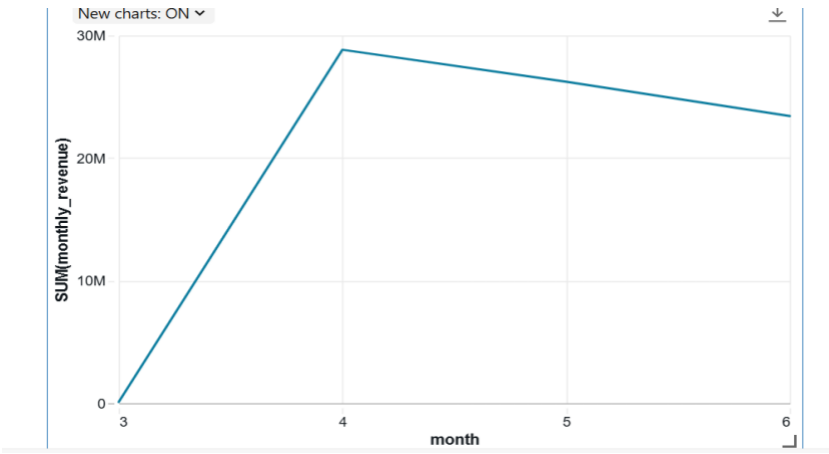
(7) Spark Jobs



Description: Further exploratory data analysis is conducted. We extract month and year from the date fields to generate a monthly sales trend line chart.

```python
#monthly sales trend
from pyspark.sql.functions import col, sum, month, year
df = amazon_df1.withColumn("amount", col("amount").cast("double"))

df = df.withColumn("year", year(col("date"))) \
        .withColumn("month", month(col("date")))

monthly_df = df.groupBy("year", "month") \
    .agg(sum("amount").alias("monthly_revenue")) \
    .orderBy("year", "month")

display(monthly_df)
```

(3) Spark Jobs



Description: To understand product performance, we perform an analysis to determine the total revenue generated by each distinct product category. Using PySpark

+ Code + Text Genie Code

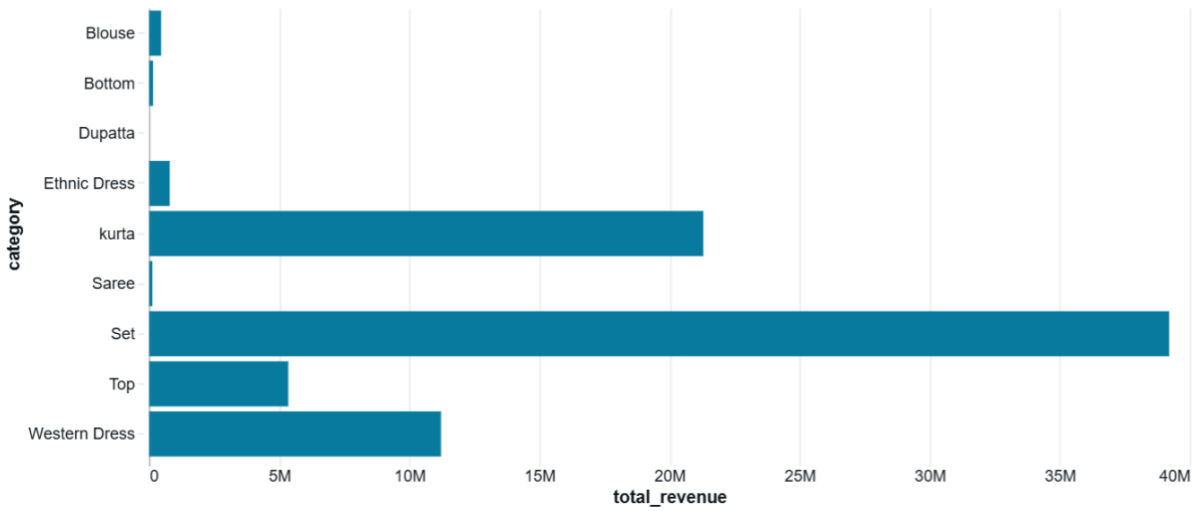
07:27 PM (7m) 25

```
from pyspark.sql.functions import col, sum

# Clean amount
df = amazon_df1.withColumn("amount", col("amount").cast("double"))

# Category-wise revenue
category_df = df.groupBy("category") \
    .agg(sum("amount").alias("total_revenue")).filter(col("total_revenue") > 0) \
    .orderBy(col("total_revenue").desc())

display(category_df)
```

(3) Spark Jobs

category_df: pyspark.sql.dataframe.DataFrame = [category: string, total_revenue: double]

df: pyspark.sql.dataframe.DataFrame = [ship_service_level: string, order_id: string ... 20 more fields]



Next, we segment orders into "High Value," "Medium Value," and "Low Value" buckets based on amount thresholds, generating bar charts to visualize the total order counts and revenue millions per segment.

08:43 PM (7m) 19 Python

```python
from pyspark.sql.functions import col, when,sum,count
# Clean amount
df = amazon_df1.withColumn("amount", col("amount").cast("double"))

# Create order value segments
order_value_df = df.withColumn(
    "order_type",
    when(col("amount") < 500, "Low Value") \
    .when((col("amount") >= 500) & (col("amount") < 2000), "Medium Value") \
    .otherwise("High Value")
)

# Count orders per segment
result_df = order_value_df.groupBy("order_type") \
    .agg(
        count("*").alias("order_count"),
        sum("amount").alias("total_revenue")
    )

display(result_df)
```

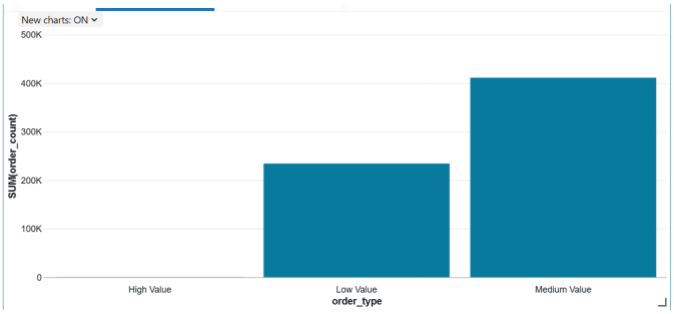
```python
result_df = result_df.withColumn(
    "revenue_millions",
    col("total_revenue") / 1000000
)

display(result_df)
```

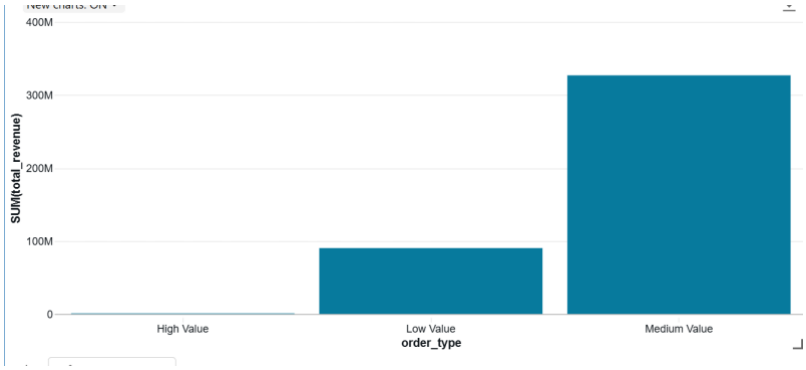
(3) Spark Jobs

result_df: pyspark.sql.dataframe.DataFrame = [order_type: string, order_count: long ... 2 more fields]

Table | Visualization 1 | Visualization 2 | +

	order_type	order_count	total_revenue	revenue_millions
1	Medium Value	411143	326959750.79999804	326.95975079999806
2	High Value	288	732074.1499999999	0.7320741499999999
3	Low Value	234145	90208863.6700001	90.20886367000011





Description: We aggregate the data to identify the Top 10 SKUs by revenue, along with their category, size, and total distinct orders, specifically for orders where the status is marked as "Shipped."

```
#Identify Top 10 SKUs by revenue (Sales amount), along with their category, size, and number of orders – for Shipped
orders.
from pyspark.sql.functions import sum, countDistinct

top_skus = amazon_clean.filter(col("Status") == "Shipped") \
    .groupBy("SKU", "Category", "Size") \
    .agg(
        countDistinct("order_id").alias("orders"),
        sum("Amount").alias("revenue")
    ) \
    .orderBy(col("revenue").desc())

display(top_skus.limit(10))
```

(4) Spark Jobs

	SKU	Category	Size	orders	revenue
1	j0230-skd-m	Set	M	385	2337489
2	j0230-skd-s	Set	S	337	2082797
3	set268-kr-np-xl	Set	XL	336	1444492
4	set268-kr-np-l	Set	L	317	1351646
5	set268-kr-np-s	Set	S	312	1347206
6	j0230-skd-l	Set	L	216	1295045
7	j0230-skd-xl	Set	XL	184	1130067
8	j0230-skd-xs	Set	XS	180	1110810
9	jne3405-kr-l	kurta	L	422	939375
10	set278-kr-np-...	Set	M	111	829866

10 rows | 3m 17s runtime

Refreshed 13 minutes ago

Phase 5: Materialized Views and SQL Database Setup

1. To serve the transformed and aggregated data to external reporting tools, we set up an Azure SQL Database for storing materialized views. The Mydb SQL database is provisioned. The cleaned DataFrames from Databricks will be loaded into this Azure SQL Database to optimize querying for final dashboard reporting